GOLD: Building Real Classes (Wed Sep 23)

Static vs Non-static Methods (from Monday)

static vs non-static methods

We've used both kinds of methods before: `Math.pow(2, 5)` is called on the class, `s.length()` on an object.

Now we'll look at how to write both.

```
class Main {
    void main() {
        Wheel a = new Wheel(1.0);
        Wheel b = new Wheel(2.0);
        IO.println(b.getCircumference());     // instance: needs an object; uses b's r
        IO.println(Wheel.getWheelCount());    // static: ask the CLASS; no object need
    }
}

class Wheel {
    double radius;
    static int wheelCount;

    Wheel(double radius) {
        this.radius = radius;
        wheelCount++;
    }

    double getCircumference() {          // instance method: has a "this"
        return 2 * Math.PI * this.radius;
    }

    static int getWheelCount() {         // static method: no "this", static data only
        return wheelCount;
    }
}
```

- **Instance method:** belongs to an object.
  - It has a `this` available, so it can read and write *that object's* fields.
  - `getCircumference()` uses `this.radius`.

- **Static method:** belongs to the class.
  - There is **no `this`**, so it can touch only static data and its own parameters.
  - No object is needed to call it.

This is also why the getter for a static field is itself static: you ask the class using `Wheel.getWheelCount()`, not any one wheel.

Pause and predict

```
class Wheel {
    double radius;

    static double area() {
        return Math.PI * radius * radius;
    }
}
```

What happens?

- **A.** It compiles. `area()` uses the radius of the most recently created Wheel
- **B.** Compile error: non-static variable `radius` cannot be referenced from a static context
- **C.** It compiles, and `radius` is `0.0` inside a static method
- **D.** It compiles, but crashes at run time with a `NullPointerException`

Build: a Die (from Monday)

Build: a Die

Let's build some programs for game night. First up: a die you can roll.

Design questions first for the Die class:

- What it knows (fields):
  - how many faces it has;
  - what it currently shows;
  - and where its random numbers come from.
- What it does (methods): roll.

`Random` does the rolling: `rng.nextInt(n)` gives `0` up to `n - 1`, so `1 + rng.nextInt(numFaces)` is a die.

```
import java.util.Random;

class Main {
    void main() {
        Die d6 = new Die(6);
        Die d20 = new Die(20);
        IO.println("d6:  " + d6.roll());
        IO.println("d20: " + d20.roll());
        IO.println("d6 still shows: " + d6.getState());
    }
}

class Die {
    private Random rng;      // a Die CONTAINS a Random: a field can hold an object
    private int numFaces;
    private int state;       // last roll; 0 = not yet rolled

    public Die(int numFaces) {
        this.rng = new Random();
        this.numFaces = numFaces;
        this.state = 0;
    }

    public int roll() {
        state = 1 + rng.nextInt(numFaces);
        return state;
    }

    public int getState() {
        return state;
    }
}
```

Notes:

- `roll()` is a method with real behavior: it changes the object's state *and* reports it.
- Each Die holds its own `state`: rolling `d6` doesn't touch `d20`.
- `rng` is a field that holds **an object**. A Die *contains* a Random. Classes are built using other classes.
- The file now starts with `import java.util.Random;`.
- Wouldn't it be nice if `new Die()` just meant a six-sided die?

Counting Objects (from Monday)

The constructor sees every creation

How would you count how many `Tree` objects a program has created?

You *could* put a counter next to every `new Tree(...)` in the program.

We will surely miss one, somewhere, eventually.

Better: every new Tree runs the constructor, so count there, in a static field. One shared counter for the whole class:

```
class Main {
    void main() {
        Tree t1 = new Tree(31.4);
        Tree t2 = new Tree(62.8);
        Tree t3 = new Tree(15.7);
        IO.println("trees created: " + Tree.getNumberOfTrees());   // 3
    }
}

class Tree {
    private double circumference;
    private static int numberOfTrees = 0;

    public Tree(double circumference) {
        this.circumference = circumference;
        numberOfTrees++;              // the constructor sees every birth
    }

    public double getRadius() {
        return circumference / (2 * Math.PI);
    }

    public static int getNumberOfTrees() {
        return numberOfTrees;
    }
}
```

Two things to notice:

- The bookkeeping is **centralized**: no call site can forget, because construction is the count.
- The getter for a static field is itself static: you ask the class, `Tree.getNumberOfTrees()`, not any

one tree.

The Convention and the Contract (from Monday)

The convention, and the contract

The convention

You'll notice that in `Die` and `Tree` the fields are `private` and the methods are `public`.

That is the standard shape of a Java class:

- **fields `private`**: outsiders can't reach in
- **methods `public`**: outsiders use these as the "front door"

Why? We'll explain next time.

The contract

Autograded work, in labs, take-homes and exams, checks your class against its **specification**:

- exact class name,
- exact method names,
- exact parameter lists,
- and the return types.

Read the spec like a contract:

1. Write every field and every method header **first**, straight from the spec, before any bodies.
2. Compile. Fix names and types until it is exactly the spec.
3. *Then* fill in the bodies.

Call by Value

How are arguments handed to a method?

Java uses **call by value**:

- the **value** of each **argument** is **copied** into the corresponding parameter.
- We saw this for arrays: the parameter receives a copy of the arrow.
- It is the same story for every object

A method can't change a caller's `int`

```
class Main {
    void changer(int x) {
        x = 12;                 // changes the LOCAL copy
    }

    void main() {
        int y = 5;
        changer(y);
        IO.println(y);          // 5: unchanged!
    }
}
```

`changer` got a *copy* of `y`; changing `x` can't touch `y`.

But it *can* change a caller's object

Objects are reached through **references**. The reference is copied, but the copy points at the **same object**:

```
class Main {
    void modify(Wheel wagon) {
        wagon.setRadius(22.6);      // changes the shared object
    }

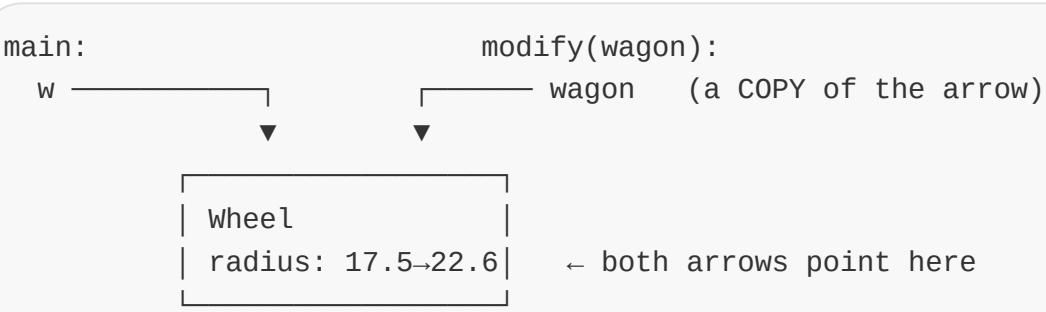
    void main() {
        Wheel w = new Wheel(17.5);
        IO.println(w.getRadius());  // 17.5
        modify(w);
        IO.println(w.getRadius());  // 22.6: changed!
    }
}

class Wheel {
    private double radius;

    Wheel(double radius) {
        this.radius = radius;
    }

    double getRadius() {
        return radius;
    }

    void setRadius(double radius) {
        this.radius = radius;
    }
}
```



- The arrow in `w` is copied into `wagon`. This is call by value.
- Both arrows point at the **same** `Wheel`, so a change made through either name is visible through both.
- Notice that `modify` goes through `setRadius`. The field is `private` (more details later)

And what does `==` mean for objects?

Two arrows can share one object. `==` compares arrows (identity), never contents:

```
class Main {
    void main() {
        Wheel w1 = new Wheel(17.5);
        Wheel w2 = new Wheel(17.5);
        Wheel w3 = w1;
        IO.println(w1 == w2);   // false: two objects, same contents
        IO.println(w1 == w3);   // true:  one object, two names
    }
}

class Wheel {
    private double radius;

    Wheel(double radius) {
        this.radius = radius;
    }
}
```

Just like Strings and arrays: `.equals` exists because `==` answers “same object?”

Pause and predict (poll)

Q. `Wheel` is the class from the `modify` example above (constructor, `getRadius`, `setRadius`). What prints?

```
class Main {
    void tweak(Wheel w, double r) {
        w.setRadius(r);
        w = new Wheel(99.0);
        w.setRadius(1.0);
    }

    void main() {
        Wheel a = new Wheel(17.5);
        tweak(a, 42.0);
        IO.println(a.getRadius());
    }
}
```

- **A.** `17.5`
- **B.** `42.0`
- **C.** `99.0`
- **D.** `1.0`

Scope

Scope

Scope is the region of code where a variable name can be used.

Basic rule: a variable may be used **from its declaration to the end of its enclosing block** `{ }`.

```
class Main {
    void main() {
        int a = 1;                // a: usable to the end of main
        if (a > 0) {
            int b = 2;            // b: usable only inside this if-block
            IO.println(a + b);
        }
        // b is out of scope here: using it would not compile
        for (int i = 0; i < 3; i++) {
            IO.println(i);        // i usable here
        }
        // i is out of scope here too
    }
}
```

Shadowing: a parameter hides a field

When a parameter (or local variable) has the **same name** as a field, the field is **shadowed**.

- Use `this.` to reach it.
- This is the only case where a name conflict is allowed.
- This is why `radius = radius;` did nothing on Monday: both names meant the parameter.

```
class Main {
    void main() {
        new Shadow().show(99);
    }
}

class Shadow {
    int x = 10;                    // field

    void show(int x) {             // parameter x SHADOWS the field
        IO.println(x);             // 99: the parameter
        IO.println(this.x);        // 10: the field
    }
}
```

No duplicate names in a scope

You **cannot** declare two variables with the same name active in the same scope:

```
void bad() {
    int x = 1;
    int x = 2;        // COMPILE ERROR: variable x is already defined
}

void ok() {
    { int x = 1; }    // fine: separate block
    { int x = 2; }    // fine: separate block
}
```

Shadowing a *field* is the one allowed exception: a parameter or local may reuse a field's name.

Overloading

Overloading: same name, different signatures

A method's **signature** is its *name* plus its *parameter types* (not including the return type).

A class may have several methods with the **same name** if their signatures differ:

```
double getPayment(int months, double rate)    // signature A
double getPayment(int years)                  // signature B
double getPayment()                           // signature C
```

This practice is called **overloading**.

- Java picks which one of the methods to call by matching the **argument types**.
- A familiar example: `IO.println(int)`, `IO.println(String)`, `IO.println(double)` ...

Overloading in action

Suppose we want to generalize the `isPalindrome` method we wrote for `String`s to work for `int`s as well:

- We could solve the problem from scratch to compare digits
- Or....

```
class Main {
    boolean isPalindrome(String s) {
        if (s == null || s.length() <= 1) {
            return true;
        }
        while (s.length() > 1) {
            if (s.charAt(0) != s.charAt(s.length() - 1)) {
                return false;
            }
            s = s.substring(1, s.length() - 1);
        }
        return true;
    }

    boolean isPalindrome(int x) {                // overload: takes an int
        return isPalindrome(Integer.toString(x)); // reuse the String version
    }

    void main() {
        IO.println(isPalindrome("racecar"));     // true
        IO.println(isPalindrome(12321));         // true
        IO.println(isPalindrome(123));           // false
    }
}
```

More examples:

```
int    calc(double x, int y)          // the original
int    calc(double x, int y, int z)   // valid: more parameters
double calc(double x)                 // valid: fewer parameters
int    calc(double x, String s)       // valid: different types
double calc(double a, int b)          // INVALID: only the return type differs
```

What is *not* a valid overload

Overloads must differ in their **signature**, and the **return type is not part of it**:

```
int    calc(double a, int b) { return b; }
double calc(double a, int b) { return a; }   // COMPILE ERROR:
                                             // method calc(double,int) is already def
```

Changing only the return type is **not** enough: Java can't tell from the call which one you meant.

Pause and predict (poll)

Q. Given `int calc(double x, int y)`, which is a **valid** overload?

- **A.** `double calc(double a, int b)`
- **B.** `int calc(double x)`
- **C.** `int calc(double y, int x)`
- **D.** None of these

this(): Constructor Chaining

`this(...)`: one constructor calling another

Inside a constructor, **`this(...)`** calls *another constructor of the same class*.

- Put it **first** in the constructor body: that is what almost all Java code you will read requires.
- The newest Java relaxes this rule slightly. Don't rely on it.
- You cannot call another constructor of the same class by its name, only by `this(...)`

Great for avoiding duplicated setup:

```
class Main {
    void main() {
        PurdueStudent p1 = new PurdueStudent(1010337138);
        PurdueStudent p2 = new PurdueStudent("Drake", 1123441245);
        p1.printStudent();      // (no name): 1010337138
        p2.printStudent();      // Drake: 1123441245
    }
}

class PurdueStudent {
    private boolean hasName;
    private String name;
    private int puid;

    PurdueStudent(int puid) {                // the "base" constructor
        this.puid = puid;
        hasName = false;
    }

    PurdueStudent(String name, int puid) {
        this(puid);                          // reuse the constructor above
        this.name = name;
        hasName = true;
    }

    void printStudent() {
        if (hasName) {
            IO.println(name + ": " + puid);
        } else {
            IO.println("(no name): " + puid);
        }
    }
}
```

This practice is called **constructor chaining**.

A Standard Die Constructor

We wished that `new Die()` could just mean a six-sided die.

Now it can: a no-argument `Die()` delegates to `Die(6)`.

```
import java.util.Random;

class Main {
    void main() {
        Die plain = new Die();          // now this works: a six-sided die
        Die d20 = new Die(20);
        IO.println(plain.roll() <= 6);  // true
        IO.println(d20.roll() >= 1);    // true
    }
}

class Die {
    private Random rng;
    private int numFaces;
    private int state;

    public Die(int numFaces) {          // the base constructor: all the setup lives h
        this.rng = new Random();
        this.numFaces = numFaces;
        this.state = 0;
    }

    public Die() {
        this(6);                        // "a plain die is a six-sided die"
    }

    public int roll() {
        state = 1 + rng.nextInt(numFaces);
        return state;
    }
}
```

- The setup logic lives in only one place
- the convenience constructor just delegates.
- **Principle: constructor overloads should delegate.**

Encapsulation

CS principle: encapsulation

Encapsulation means information hiding: a form of "need to know."

It serves two purposes:

- It hides implementation details from other programmers, so the author can change them without affecting anyone else's code.
- It prevents other programmers from modifying certain fields, or calling certain methods, in ways that would leave the object in an inconsistent or unexpected state.

Java access modifiers

They apply to members (fields and methods) and control access from methods in *other* classes. From least to most restrictive:

Keyword	Who can use the member
`public`	any code, anywhere
`protected`	the class, its subclasses, and classes in the same package
[none]	the class and classes in the same package ("package-private")
`private`	**only this class**

Fields become **`private`**; the outside world touches them only through **methods** you control. Two payoffs:

- **Freedom to change:** you can rework the internals without breaking other people's code.
- **Safety:** outsiders can't leave your object in a broken or inconsistent state.

```
class PurdueStudent {
    // ... other fields omitted ...
    private int birthMonth;
    private int birthDay;
    private int birthYear;

    public void setBirthdate(int mo, int day, int yr) {
        // can validate all three components together:
        // setBirthdate(2, 31, 2005) shouldn't succeed
    }
}
```

Conventional wisdom

- Make **methods** `public`: anyone may use them, and they are written defensively to protect the object's internal state.
- Make **fields** `private`: only your methods can modify them.
- Constants (`final` fields) may be `public`, since they cannot be changed anyway.

The Payoff: a Guarded Wheel

The payoff: a guarded Wheel

Here is encapsulation actually *doing* something.

- The field is private; the setter **refuses bad values**;
- the constructor sends its argument through the same guard in `setRadius`, so a Wheel is sensible from birth:

```
class Main {
    void main() {
        Wheel w = new Wheel(17.5);
        IO.println(w.getCircumference());   // fine: 109.95574287564276
        w.setRadius(-40);                   // compiles, and the guard ignores it
        IO.println(w.getRadius());          // still 17.5
        Wheel bad = new Wheel(-5);          // the constructor guards too
        IO.println(bad.getRadius());        // 1.0, the fallback
    }
}

class Wheel {
    private double radius;

    public Wheel(double radius) {
        setRadius(radius);                  // send the constructor through the same g
        if (this.radius == 0.0) {
            this.radius = 1.0;              // nothing sensible was given: fall back,
        }
    }

    public double getCircumference() {
        return 2 * Math.PI * radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if (r > 0 && r <= 1000) {           // no negative wheels, no absurd wheels
            radius = r;
        }
    }
}
```

And from *outside* the class, the compiler enforces the encapsulation:

```
class Main {
    void main() {
        Wheel w = new Wheel(17.5);
        double circ = w.getCircumference();   // fine
        double rad = w.radius;                // COMPILE ERROR: radius has private acc
    }
}

class Wheel {
    private double radius;

    public Wheel(double radius) {
        this.radius = radius;
    }

    public double getCircumference() {
        return 2 * Math.PI * radius;
    }
}
```

```
error: radius has private access in Wheel
```

That's the trade: because *only* `setRadius` can touch `radius`, the class can **promise** that the radius is always sensible.

Without `private`, it can't keep that promise.

Design note: this setter *silently* disregards a bad value. That is one choice.

Another option is to return a `boolean` saying whether the change was accepted, the way `addDie` does in the extra reading.

A third option is to cause an exception to get the caller's attention.

Say which one you chose in the method's Javadoc.

Pause and predict (poll)

Q. This `Wheel` has the same guard as above, but its constructor is written differently. What gets printed?

```
class Main {
    void main() {
        Wheel w = new Wheel(-5);
        w.setRadius(0);
        IO.println(w.getRadius());
    }
}

class Wheel {
    private double radius;

    public Wheel(double radius) {
        this.radius = radius;               // straight to the field
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if (r > 0 && r <= 1000) {
            radius = r;
        }
    }
}
```

- **A.** `-5.0`
- **B.** `0.0`
- **C.** `1.0`
- **D.** Compile error: `radius` has private access in `Wheel`

Card: final Fields and toString

Game night, part two: a playing card

Idea 1: `final` fields

A card never changes what it is.

Mark the fields `final` and Java enforces it:

- `final` fields must be assigned **exactly once, in the constructor**,
- and can never change after:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c.getRank() + " of " + c.getSuit());
        IO.println(c);                      // predict!
    }
}

class Card {
    private final String rank;   // "Ace", "2", ..., "King"
    private final String suit;   // "Hearts", "Spades", ...

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String getRank() {
        return rank;
    }

    public String getSuit() {
        return suit;
    }
}
```

A card cannot change, so we have getters, but no setters.

An object like this is **immutable**, like `String`.

Compare the Die: `roll()` *mutates* its state. Both designs are correct for their objects.

Note: a setter on a final field is a compile error:

```
public void setRank(String r) {
    this.rank = r;      // COMPILE ERROR: cannot assign a value to final variable rank
}
```

Idea 2: what does printing an object do?

Run the snippet above and look at its last line.

`IO.println(c)` printed something like `Card@7344699f`: the class name and a value called a hash.

- This is Java's default description of an object, and it is useless usually.
- The fix: give the class a `toString()` method.
- `println` and String concatenation call `toString()` automatically:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c);                    // Ace of Spades
        IO.println("drew: " + c);         // drew: Ace of Spades
    }
}

class Card {
    private final String rank;
    private final String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }
}
```

`toString` is the object's self-description. Every class you write should have one.

Pause and predict (poll)

```
Card c = new Card("Ace", "Spades");
IO.println(c);
```

`Card` has no `toString` method. What prints?

- **A.** A compile error
- **B.** Something like `Card@7344699f`
- **C.** `Ace of Spades` anyway
- **D.** `null`

The Classic main

The classic `main`

```
public static void main(String[] args) { … }
```

word	meaning
`public`	any code anywhere may call it: the Java launcher lives outside your class
`static`	belongs to the class, not to an object: when the program starts, **no object exists yet**, so the launcher can only call a static method
`void`	returns nothing
`main`	the name the launcher looks for
`String[] args`	an array of the words typed after the program name on the command line

```
class Main {
    public static void main(String[] args) {
        IO.println("classic main: args.length = " + args.length);
        for (int i = 0; i < args.length; i++) {
            IO.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Run as `java Main add 5 17.25`, this prints `args.length = 3`, then `add`, `5` and `17.25`.

Everything in `args` is a String: `args[1]` is `"5"`, not `5`.

Pressing Run on a slide supplies no arguments, so `args.length` is 0.

Our `void main()` is the same method without the boilerplate:

- Java builds an object of your class first and calls `main` on it
- That's why `this.` has been available all semester.

- Both forms are correct.

Extent

Scope vs. extent (lifetime)

- **Scope:** *where* is the name usable? (a compile-time region)
- **Extent:** *how long* does the value live? (a run-time lifetime)

For a value stored in a **local variable**, extent = scope:

- it is born at its declaration
- and dies at the block's `}`:

```
class Main {
    void main() {
        int pass = 0;
        while (pass < 3) {
            int i = 0;      // re-created and re-initialized EACH pass
            i++;
            IO.println(i);  // always prints 1
            pass++;
        }
    }
}
```

Object extent: alive while reachable

An **object** lives as long as *something* still references it.

When the last reference is gone, it becomes eligible for **garbage collection**:

```
String s = new String("Purdue");   // object P
String t = s;                      // s and t both point at P
s = new String("Indiana State");   // s → new object; P still held by t
t = new String("Ball State");      // t → new object; nothing holds P now
```

```
after line 2:   s → "Purdue" ← t
after line 3:   s → "Indiana State"      t → "Purdue"
after line 4:   s → "Indiana State"      t → "Ball State"
```

`"Purdue"` is now unreachable and eligible for garbage collection.

You never delete objects by hand in Java; you just stop pointing at them.

Extra Reading: Scope Demo

Extra reading: predict the output, then trace it. Three blocks, three different `x`s.

```
public class Demo {

    public static void doStuff() {

        for (int i=0; i < 10; i++) {
            System.out.println(i);
        }

        for (int i=10; i > 0; i--) {
            System.out.println(i);
        }

        int y = 0;

        {
            int x = 10;
            y += x;
        }

        {
            int x = 20;
            y += x;
        }

        {
            int x = 40;
            y += x;
        }

        System.out.println(y);
    }

    public static void main(String[] args) {
        doStuff();
    }
}
```

Extra Reading: A Cup of Dice

A cup of Dice: an array of YOUR objects

Extra reading: not covered in lecture, and its code is shown rather than runnable. Friday's workshop builds this.

You can make an array of `int`s and an array of `String`s, so of course you can also make an array of `Die`.

```
public class DiceCup {
    private Die[] dice;      // capacity
    private int count;       // how many are actually in the cup

    public DiceCup(int capacity) {
        dice = new Die[capacity];   // all slots null: no Die objects yet!
        count = 0;
    }

    public boolean addDie(Die d) {
        if (count == dice.length)
            return false;           // full: refuse, and say so
        dice[count] = d;
        count++;
        return true;
    }

    public int rollAll() {
        int total = 0;
        for (int i = 0; i < count; i++)   // count, not dice.length!
            total += dice[i].roll();
        return total;
    }

    public static void main(String[] args) {
        DiceCup cup = new DiceCup(3);
        cup.addDie(new Die(6));
        cup.addDie(new Die(6));
        IO.println("total: " + cup.rollAll());
        IO.println(cup.addDie(new Die(20)));  // true: it fits
        IO.println(cup.addDie(new Die(20)));  // false: it's full
    }
}
```

Notes:

- `new Die[capacity]` makes **null slots**, not Dice. Objects only exist after `new Die(...)` fills a slot.
- `capacity` = array size. `count` = how many dice there really are.
- **Which one is real?** `count` . The `capacity` is an implementational detail, not a reflection of something in the real world.
- **Why `i < count` and not `i < dice.length`?** Because slot 2 is still `null`

- `addDie` returns `boolean`: a full cup refuses and says so instead of crashing.

- Removing something is the same idea in reverse: the slot becomes available again (set it to `null`, or move another element into it) and `count` goes down.

Extra Reading: Deck

Deck: an array of Cards

Extra reading: not covered in lecture, and its code is shown rather than runnable. Friday's workshop builds this.

Now a Deck: an array of your objects, plus a count.

```java
public class Deck {
    private Card[] cards;    // capacity: 52 slots
    private int count;       // how many are actually there

    public Deck() {
        cards = new Card[52];
        count = 0;
        String[] ranks = {"Ace", "2", "3", "4", "5", "6", "7",
                          "8", "9", "10", "Jack", "Queen", "King"};
        String[] suits = {"Hearts", "Spades", "Diamonds", "Clubs"};
        for (String suit : suits)
            for (String rank : ranks)
                cards[count++] = new Card(rank, suit);
    }

    public int cardsLeft() {
        return count;
    }

    public Card deal() {
        if (count == 0)
            return null;          // "no answer": nothing left to deal
        count--;
        return cards[count];
    }

    public static void main(String[] args) {
        Deck deck = new Deck();
        IO.println("cards: " + deck.cardsLeft());   // 52
        IO.println(deck.deal());                    // toString at work
        IO.println(deck.deal());
        IO.println("cards: " + deck.cardsLeft());   // 50
    }
}
```

Code notes:

- `new Card[52]` makes 52 **null** slots: no Cards exist yet.
- The nested loop **fills** them with `new`, and `count` tracks how many are real.
- `deal()` takes from the **end** (`cards[count - 1]`). No shifting: the array slot beyond `count` simply stops mattering.
- **The returning-`null` convention:** when a method has *no answer to give* (the deck is empty, the thing wasn't found) returning `null` is the standard signal.

Extra Reading: Varargs

varargs: a method that takes any number of arguments

Optional reading: not needed for the labs or tests.

Sometimes you want to have flexibility: you may want to write a method to accept *any* count of arguments. Use `...`:

```
static int sum(int... nums) {     // nums acts like an int[]
    int total = 0;
    for (int n : nums) total += n;
    return total;
}

void main() {

    IO.println(sum(1, 2, 3));    // 6
    IO.println(sum());           // 0
    IO.println(sum(10, 20));     // 30
}
```

One rule: the `...` parameter must be **last**.

Examples you've used:

- `String.format(...)` and
- `printf(...)` are varargs methods.

Extra Reading: StringBuilder

StringBuilder: the mutable String

Extra reading. Optional bonus material

`String` objects are **immutable**.

Every operation that would change a String makes a new object.

To build text piece by piece, use a **`StringBuilder`** object:

```
void main() {
    StringBuilder sb = new StringBuilder();
    sb.append("Boiler").append(" ").append("Up").append('!');
    IO.println(sb.toString());    // Boiler Up!
    IO.println(sb.length());      // 10
}
```

Take Home: TimeOfDay

A `TimeOfDay` is a clock time: an `hour` from 0 to 23 and a `minute` from 0 to 59.

The class must **never hold an impossible time**.

Open `TimeOfDay.java` and complete it.

- **Three constructors** that chain with `this(...)`:
  - `TimeOfDay(int hour, int minute)` does the work,
  - `TimeOfDay(int hour)` means on the hour,
  - and `TimeOfDay()` means midnight.
  - Write the setup once; the other two delegate.

- **Guarded setters.** `setHour` stores only 0 to 23; `setMinute` stores only 0 to 59. A bad value is ignored and the time does not change.

- **A guarded birth.** The two-argument constructor should send its values through the setters, so that `new TimeOfDay(24, -1)` comes out as `00:00`, not as nonsense.

- **`toString()`** in 24-hour form with leading zeros: `09:05`, `23:59`. `String.format("%02d", 9)` gives `09` (Week 2's format strings, back again).

Two files, as before: `TimeOfDay.java` is yours; `Main.java` uses it, and **Run** runs `Main`.

Press **Test** to run the tests.